

GPUMODE Lecture Study Notes: Lecture 83: Formalized Kernel Derivation

Core Problem the Lecture Addresses

The central problem of **formalized kernel derivation** is how to move from a high-level tensor expression to a correct, efficient GPU kernel in a disciplined way. In many GPU programming workflows, kernels are written by intuition: choose a block size, write index math, add shared memory, tune, profile, and repeat. This can work, but it is fragile. A small indexing mistake, an incorrect boundary condition, a bad memory layout assumption, or a hidden race condition can silently produce wrong results.

The lecture's likely focus is the following engineering question:

How can we derive a GPU kernel from a mathematical specification while preserving correctness at every transformation step and exposing enough structure to optimize memory movement, parallelism, and arithmetic intensity?

This matters because modern deep learning systems depend on kernels that are simultaneously:

- **mathematically correct**, matching the tensor program specification;
- **hardware efficient**, using coalesced memory access, tiling, vectorization, shared memory, registers, and tensor cores;
- **shape general**, supporting realistic tensor dimensions and boundary cases;
- **composable**, allowing kernels to be generated, transformed, fused, and verified;
- **maintainable**, so optimization does not destroy the connection to the original program.

A formal derivation treats a kernel not as handwritten magic but as the endpoint of a sequence:

Tensor Specification $\rightarrow$ Loop Nest $\rightarrow$ Iteration Space Mapping $\rightarrow$ Memory Schedule $\rightarrow$ Parallel GPU Kernel $\rightarrow$

Each arrow should preserve semantics. Performance transformations such as tiling, fusion, vectorization, reordering, and caching must be justified by equivalence rules or dependency analysis.

Required Background

Tensor Programs as Mathematical Specifications

A tensor program describes outputs as functions of inputs. For example, matrix multiplication is specified as

$$C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} B_{k,j},$$

where

- $A \in \mathbb{R}^{M \times K}$,
- $B \in \mathbb{R}^{K \times N}$,
- $C \in \mathbb{R}^{M \times N}$,
- $i \in [0, M)$,
- $j \in [0, N)$,
- $k \in [0, K)$.

The specification says what must be computed, but it does not say how to map the computation to threads, warps, shared memory, or registers.

Loop Nests

A direct implementation of matrix multiplication is a loop nest:

```
for (int i = 0; i < M; ++i) {
    for (int j = 0; j < N; ++j) {
        float acc = 0.0f;
        for (int k = 0; k < K; ++k) {
            acc += A[i * K + k] * B[k * N + j];
        }
        C[i * N + j] = acc;
    }
}
```

This loop nest is a concrete operational form of the equation. It introduces an execution order, a memory layout assumption, and an accumulation variable. Formalized derivation asks whether transformations of this loop nest preserve the mathematical meaning of the original tensor expression.

GPU Execution Model

A CUDA kernel executes over a hierarchy:

- **Grid:** collection of thread blocks.
- **Thread block:** collection of threads that can synchronize and share data through shared memory.
- **Warp:** group of usually 32 threads executed in SIMT fashion.
- **Thread:** scalar execution context with private registers.

The mapping from logical indices to GPU execution indices is usually expressed through

$$\text{global thread id} = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

For two-dimensional problems, we often use

$$i = \text{blockIdx.y} \cdot \text{blockDim.y} + \text{threadIdx.y},$$

$$j = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

This mapping is a formal relationship between the mathematical iteration space and the physical execution space.

Memory Hierarchy

A GPU kernel interacts with several memory levels:

- **Global memory:** large, high latency, high bandwidth, shared across the device.
- **L2 cache:** shared cache across SMs.
- **L1 cache:** per-SM cache on many architectures.
- **Shared memory:** explicitly managed per-block memory.
- **Registers:** per-thread storage, fastest but limited.

Formal derivation becomes performance relevant when it can reason about data reuse. For example, a tile of A reused across many columns of C should be moved once from global memory and reused from shared memory or registers.

Main Concepts

From Specification to Iteration Space

A tensor expression defines an **iteration space**. For matrix multiplication, the output iteration space is

$$\mathcal{I}_C = \{(i, j) \mid 0 \leq i < M, 0 \leq j < N\}.$$

The reduction space for each output element is

$$\mathcal{R}_{i,j} = \{k \mid 0 \leq k < K\}.$$

The complete computation space is therefore

$$\mathcal{D} = \{(i, j, k) \mid 0 \leq i < M, 0 \leq j < N, 0 \leq k < K\}.$$

A formal derivation starts by making this domain explicit. Once the domain is explicit, transformations can be reasoned about algebraically.

Dependence and Legality of Transformations

A loop transformation is legal if it preserves program dependencies. For a pure elementwise operation,

$$Y_i = f(X_i),$$

each output Y_i is independent. Therefore, iterations may be reordered, parallelized, vectorized, or split freely.

For reductions,

$$Y_i = \sum_{k=0}^{K-1} X_{i,k},$$

the order of accumulation is mathematically associative over real numbers, but floating-point arithmetic is not perfectly associative:

$$(a + b) + c \neq a + (b + c)$$

in general for floating-point values. Thus, a formally derived kernel should distinguish between

- **real-number semantics**, where associativity is exact;

- **floating-point semantics**, where accumulation order can change numerical results.

For performance engineering, many kernels accept small numerical differences in exchange for parallel reduction. However, the derivation should explicitly state this assumption.

Schedule Versus Algorithm

The **algorithm** specifies what is computed. The **schedule** specifies how it is computed.

For matrix multiplication, the algorithm is

$$C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} B_{k,j}.$$

A schedule may choose to

- tile i , j , and k ;
- map tile indices to thread blocks;
- map intra-tile indices to warps and lanes;
- cache A and B tiles in shared memory;
- accumulate partial results in registers;
- vectorize global memory loads;
- use tensor cores for matrix multiply accumulate operations.

A key idea is that schedule choices should not change the mathematical result, except for explicitly acknowledged floating-point reorder effects.

Index Decomposition

A common formal step is decomposing a loop index into tile and intra-tile components. Suppose we tile index i by tile size T_i . Then

$$i = i_o T_i + i_i,$$

where

$$0 \leq i_o < \left\lceil \frac{M}{T_i} \right\rceil, \quad 0 \leq i_i < T_i.$$

The original bound $0 \leq i < M$ becomes

$$i_o T_i + i_i < M.$$

This boundary predicate is essential for correctness when M is not divisible by T_i .

For two-dimensional tiling,

$$i = i_o T_i + i_i, \quad j = j_o T_j + j_i.$$

The output tile domain is

$$(i_o, j_o) \in \left[0, \left\lceil \frac{M}{T_i} \right\rceil\right) \times \left[0, \left\lceil \frac{N}{T_j} \right\rceil\right).$$

Mapping Tiles to CUDA Blocks

A tiled matrix multiplication commonly maps one output tile to one CUDA thread block:

$$\text{blockIdx.y} = i_o, \quad \text{blockIdx.x} = j_o.$$

The block computes a tile

$$C[i_o T_i : i_o T_i + T_i, j_o T_j : j_o T_j + T_j].$$

Inside the block, threads cooperatively compute elements or subtiles of that output tile. The formal mapping must specify which thread computes which element or microtile.

Thread-Level Microtiles

Instead of assigning one output element per thread, optimized kernels often assign a small register tile per thread. For example, one thread may compute

$$C_{i+r, j+s} \quad \text{for } 0 \leq r < R_i, 0 \leq s < R_j.$$

The thread accumulates a register fragment

$$\text{acc}_{r,s} = \sum_{k=0}^{K-1} A_{i+r,k} B_{k,j+s}.$$

This improves data reuse because one loaded value from A can be reused across multiple s values, and one loaded value from B can be reused across multiple r values.

Hardware-Level Explanation

Why Formal Mapping Matters on GPUs

GPUs execute massive numbers of threads, but performance depends heavily on how those threads access memory and share work. A mathematically correct mapping can still be slow if it causes:

- non-coalesced global memory loads;
- redundant global memory traffic;
- excessive register pressure;
- low occupancy;
- shared memory bank conflicts;
- warp divergence;
- poor tensor core utilization;
- unbalanced work across thread blocks.

Formalized derivation helps because it makes the mapping explicit. Once the mapping is explicit, we can reason about whether adjacent lanes access adjacent addresses, whether data is reused, and whether synchronization is required.

Memory Coalescing

For a row-major array X with shape $M \times N$, the address of $X_{i,j}$ is

$$\text{addr}(X_{i,j}) = \text{base}(X) + (iN + j) \cdot \text{sizeof}(X_{0,0}).$$

If consecutive lanes in a warp access

$$X_{i,j}, X_{i,j+1}, X_{i,j+2}, \dots, X_{i,j+31},$$

then addresses are contiguous. This is coalesced and efficient.

If consecutive lanes access

$$X_{i,j}, X_{i+1,j}, X_{i+2,j}, \dots, X_{i+31,j},$$

then addresses differ by a stride of N . This is usually less efficient for row-major layout.

Thus, formalized derivation must include layout-aware index mapping.

Shared Memory Tiling

In tiled matrix multiplication, a block loads tiles

$$A_{\text{tile}} \in \mathbb{R}^{T_M \times T_K}, \quad B_{\text{tile}} \in \mathbb{R}^{T_K \times T_N}.$$

The block then computes

$$C_{\text{tile}} += A_{\text{tile}} B_{\text{tile}}.$$

The reason this is faster than the naive implementation is reuse. Each element of A_{tile} may be reused by T_N output columns, and each element of B_{tile} may be reused by T_M output rows.

Global memory traffic per k tile is approximately

$$T_M T_K + T_K T_N$$

input elements, while the number of multiply-add operations is approximately

$$2T_M T_N T_K$$

floating-point operations.

The arithmetic intensity of the tile is therefore

$$\text{AI} = \frac{2T_M T_N T_K}{(T_M T_K + T_K T_N + T_M T_N) \cdot b},$$

where b is the number of bytes per element and the output write cost $T_M T_N$ is included once per tile result.

Synchronization

When threads cooperatively load shared memory, a barrier is required before any thread consumes the tile:

```
shared_A[thread_y][thread_x] = A[global_a_index];
shared_B[thread_y][thread_x] = B[global_b_index];

__syncthreads();

for (int kk = 0; kk < TILE_K; ++kk) {
    acc += shared_A[local_i][kk] * shared_B[kk][local_j];
}

__syncthreads();
```

The first synchronization ensures all loads are visible before computation. The second synchronization ensures no thread overwrites shared memory for the next tile before other threads finish reading the current tile.

Occupancy and Register Pressure

Occupancy measures how many warps can be resident on an SM relative to the architectural maximum. A simplified expression is

$$\text{occupancy} = \frac{\text{active warps per SM}}{\text{maximum warps per SM}}.$$

Occupancy is limited by

- threads per block;
- registers per thread;
- shared memory per block;
- architectural limits on blocks and warps per SM.

A formal derivation can estimate resource usage. For a thread microtile of shape $R_M \times R_N$, the accumulator registers alone require approximately

$$R_M R_N$$

floating-point registers per thread. Additional registers are needed for indices, pointers, loaded fragments, predicates, and loop counters. Larger microtiles improve reuse but may reduce occupancy.

Mathematical Reasoning

Correctness of Tiling

Consider the original matrix multiplication:

$$C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} B_{k,j}.$$

Choose a tile size T_K and decompose k as

$$k = k_o T_K + k_i.$$

Then

$$C_{i,j} = \sum_{k_o=0}^{\lceil K/T_K \rceil - 1} \sum_{k_i=0}^{T_K-1} \mathbf{1}_{k_o T_K + k_i < K} A_{i,k_o T_K + k_i} B_{k_o T_K + k_i, j}.$$

The indicator

$$\mathbf{1}_{k_o T_K + k_i < K}$$

handles non-divisible dimensions. This equation shows that tiled accumulation is equivalent to the original reduction as long as the boundary predicate is respected.

Correctness of Parallel Elementwise Kernels

For an elementwise operation

$$Y_n = f(X_n), \quad 0 \leq n < N,$$

a one-dimensional CUDA mapping uses

$$n = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}.$$

The kernel is correct if every $n \in [0, N)$ is computed exactly once and no out-of-range write occurs. With grid size

$$G = \left\lceil \frac{N}{B} \right\rceil,$$

where $B = \text{blockDim.x}$, the launched domain is

$$\mathcal{L} = \{bB + t \mid 0 \leq b < G, 0 \leq t < B\}.$$

This domain covers all valid indices:

$$[0, N) \subseteq \mathcal{L}.$$

However, it may include extra indices $n \geq N$. Therefore, the kernel needs a guard:

$$n < N.$$

Roofline Model

The roofline model estimates performance using arithmetic intensity:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Moved}}.$$

The attainable performance is bounded by

$$P \leq \min(P_{\max}, B_{\max} \cdot \text{AI}),$$

where

- P is achieved throughput in FLOPs per second;
- $P_{\max}$ is peak compute throughput;
- $B_{\max}$ is peak memory bandwidth;
- AI is arithmetic intensity.

If

$$B_{\max} \cdot \text{AI} < P_{\max},$$

the kernel is memory-bound. If

$$B_{\max} \cdot \text{AI} \geq P_{\max},$$

the kernel can become compute-bound.

Work and Memory Complexity

For matrix multiplication,

$$\text{FLOPs} = 2MNK.$$

For a naive implementation, each output element reads K elements from A and K elements from B , then writes one element to C . The approximate global memory traffic is

$$\text{Bytes}_{\text{naive}} = (MN(2K) + MN)b,$$

where b is bytes per element. The arithmetic intensity is approximately

$$\text{AI}_{\text{naive}} = \frac{2MNK}{MN(2K+1)b} = \frac{2K}{(2K+1)b}.$$

For large K ,

$$AI_{\text{naive}} \approx \frac{1}{b}.$$

For FP32, $b = 4$, so

$$AI_{\text{naive}} \approx 0.25 \text{ FLOPs per byte.}$$

This is very low, so naive matrix multiplication is memory-inefficient despite having many arithmetic operations.

A tiled implementation improves reuse and increases arithmetic intensity by loading tiles once and reusing them many times.

Code Walkthrough

Naive Elementwise Kernel

```
__global__ void add_kernel(  
    const float* __restrict__ x,  
    const float* __restrict__ y,  
    float* __restrict__ z,  
    int n  
) {  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;  
  
    if (idx < n) {  
        z[idx] = x[idx] + y[idx];  
    }  
}
```

This kernel implements

$$z_i = x_i + y_i.$$

The index

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$$

maps one CUDA thread to one output element. Consecutive threads access consecutive elements of x , y , and z , so global memory accesses are coalesced when arrays are contiguous.

The guard

$$i < n$$

is required because the launch grid usually rounds up:

$$\text{gridDim.x} = \left\lceil \frac{n}{\text{blockDim.x}} \right\rceil.$$

Without the guard, extra threads would write out of bounds.

Naive Matrix Multiplication Kernel

```
__global__ void matmul_naive(
    const float* __restrict__ A,
    const float* __restrict__ B,
    float* __restrict__ C,
    int M,
    int N,
    int K
) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < M && col < N) {
        float acc = 0.0f;

        for (int k = 0; k < K; ++k) {
            float a = A[row * K + k];
            float b = B[k * N + col];
            acc += a * b;
        }

        C[row * N + col] = acc;
    }
}
```

This kernel maps one thread to one output element:

$$C_{\text{row},\text{col}} = \sum_{k=0}^{K-1} A_{\text{row},k} B_{k,\text{col}}.$$

The access to A is contiguous across k for a single thread. However, across a warp, if adjacent lanes compute adjacent columns for the same row, then all lanes load the same $A_{\text{row},k}$ at the same time. This can be broadcast or cached, but the implementation does not explicitly exploit reuse.

The access to B is coalesced across lanes when adjacent lanes have adjacent col values:

$$B_{k,\text{col}}, B_{k,\text{col}+1}, \dots, B_{k,\text{col}+31}.$$

The weakness is that A and B values are repeatedly loaded from global memory by many threads and blocks.

Tiled Matrix Multiplication Kernel

```
#define TILE 16

__global__ void matmul_tiled(
    const float* __restrict__ A,
    const float* __restrict__ B,
    float* __restrict__ C,
    int M,
    int N,
    int K
) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int local_row = threadIdx.y;
    int local_col = threadIdx.x;

    int row = blockIdx.y * TILE + local_row;
    int col = blockIdx.x * TILE + local_col;

    float acc = 0.0f;

    for (int tile_k = 0; tile_k < (K + TILE - 1) / TILE; ++tile_k) {
        int a_col = tile_k * TILE + local_col;
        int b_row = tile_k * TILE + local_row;

        if (row < M && a_col < K) {
            As[local_row][local_col] = A[row * K + a_col];
        } else {
            As[local_row][local_col] = 0.0f;
        }

        if (b_row < K && col < N) {
            Bs[local_row][local_col] = B[b_row * N + col];
        } else {
            Bs[local_row][local_col] = 0.0f;
        }
    }
}
```

```

    }

    __syncthreads();

    for (int kk = 0; kk < TILE; ++kk) {
        acc += As[local_row][kk] * Bs[kk][local_col];
    }

    __syncthreads();
}

if (row < M && col < N) {
    C[row * N + col] = acc;
}
}

```

This kernel implements the tiled decomposition

$$C_{i,j} = \sum_{k_o} \sum_{k_i} A_{i,k_o T + k_i} B_{k_o T + k_i, j}.$$

Each block computes a $TILE \times TILE$ tile of C . Threads cooperatively load one tile of A and one tile of B into shared memory.

Important hardware behavior:

- Each thread loads one element of A and one element of B per k tile.
- Shared memory allows values to be reused by multiple threads in the block.
- The first `__syncthreads()` ensures the shared memory tiles are fully loaded.
- The inner loop reuses shared memory values $TILE$ times.
- The second `__syncthreads()` prevents overwrite hazards before the next tile load.

The formal correctness condition is that zero padding for out-of-bound tile elements does not change the sum. If an invalid term is replaced by zero, then the guarded tiled sum is equivalent to the original sum.

Python Reference Implementation

```

import torch

def matmul_reference(A: torch.Tensor, B: torch.Tensor) -> torch.Tensor:
    M, K = A.shape
    K2, N = B.shape

```

```

assert K == K2

C = torch.empty((M, N), device=A.device, dtype=A.dtype)

for i in range(M):
    for j in range(N):
        acc = torch.zeros((), device=A.device, dtype=A.dtype)
        for k in range(K):
            acc = acc + A[i, k] * B[k, j]
        C[i, j] = acc

return C

```

This code is not efficient in PyTorch because Python loops serialize the computation and launch many tiny operations. However, it is useful as a specification. The formal kernel derivation should preserve the semantics of this reference while changing the schedule.

Triton-Style Blocked Matmul Sketch

```

import triton
import triton.language as tl

@triton.jit
def matmul_kernel(A, B, C, M:tl.constexpr, N:tl.constexpr, K:tl.constexpr,
                  BLOCK_M:tl.constexpr, BLOCK_N:tl.constexpr,
                  BLOCK_K:tl.constexpr):
    pid_m = tl.program_id(0)
    pid_n = tl.program_id(1)

    offs_m = pid_m * BLOCK_M + tl.arange(0, BLOCK_M)
    offs_n = pid_n * BLOCK_N + tl.arange(0, BLOCK_N)
    offs_k = tl.arange(0, BLOCK_K)

    acc = tl.zeros((BLOCK_M, BLOCK_N), dtype=tl.float32)

    for k0 in range(0, K, BLOCK_K):
        a_ptrs = A + offs_m[:, None] * K + (k0 + offs_k[None, :])
        b_ptrs = B + (k0 + offs_k[:, None]) * N + offs_n[None, :]

        a = tl.load(a_ptrs, mask=(offs_m[:, None] < M) & (k0 + offs_k[None, :] < K), other=0)
        b = tl.load(b_ptrs, mask=(k0 + offs_k[:, None] < K) & (offs_n[None, :] < N), other=0)

        acc += tl.dot(a, b)

    c_ptrs = C + offs_m[:, None] * N + offs_n[None, :]

```

```
tl.store(c_ptrs, acc, mask=(offs_m[:, None] < M) & (offs_n[None, :] < N))
```

This sketch makes the block-level derivation explicit. The program instance computes a rectangular tile of C . The offsets define vectorized index sets, and the masks encode boundary predicates.

The core operation is

$$\text{acc} += A_{\text{block}} B_{\text{block}},$$

where

$$A_{\text{block}} \in \mathbb{R}^{BLOCK_M \times BLOCK_K}, \quad B_{\text{block}} \in \mathbb{R}^{BLOCK_K \times BLOCK_N}.$$

The use of `tl.dot` tells Triton that the blocked computation may be lowered to hardware matrix multiply instructions when dtype, shape, alignment, and target architecture allow it.

Performance Intuition

Naive Kernels Are Often Bandwidth-Bound

Naive elementwise kernels perform very little computation per byte. For vector addition,

$$z_i = x_i + y_i,$$

each element performs one floating-point addition and moves approximately three values: load x_i , load y_i , and store z_i . For FP32, this is approximately

$$\text{AI} = \frac{1}{3 \cdot 4} = \frac{1}{12} \quad \text{FLOPs per byte.}$$

This is extremely memory-bound. The best optimization is usually not more arithmetic optimization, but better memory access, fusion, or avoiding unnecessary memory movement.

Tiling Converts Bandwidth Into Reuse

Matrix multiplication has enormous potential reuse. A value $A_{i,k}$ is used for many columns j , and a value $B_{k,j}$ is used for many rows i . Tiling exposes this reuse within a block.

For a tile shape $T_M \times T_N \times T_K$, the reuse factors are approximately:

$$\text{reuse}(A) \approx T_N, \quad \text{reuse}(B) \approx T_M.$$

Higher reuse increases arithmetic intensity, allowing the kernel to move from memory-bound toward compute-bound.

Thread Mapping Affects Coalescing

A correct kernel may still be slow if thread lanes map to strided memory. For row-major matrices, assigning adjacent lanes to adjacent columns usually gives coalesced access for rows.

A good derivation includes a mapping function

$$\phi : (\text{blockIdx}, \text{threadIdx}) \mapsto (i, j)$$

and then analyzes

$$\text{addr}(\phi(\ell + 1)) - \text{addr}(\phi(\ell))$$

for adjacent lanes ℓ and $\ell + 1$. If this difference is the element size b , memory access is contiguous.

Boundary Conditions Are Part of the Derivation

Many hand-written kernels are correct only when dimensions are multiples of tile sizes. A formal derivation forces boundary predicates to be part of the schedule.

For a tile index

$$i = i_o T_i + i_i,$$

the valid predicate is

$$i < M.$$

For matrix multiplication, the full predicates are

$$i < M, \quad j < N, \quad k < K.$$

These predicates must guard loads, stores, or both.

Fusion as Formal Composition

Suppose we compute

$$Y_i = f(X_i), \quad Z_i = g(Y_i).$$

The unfused version writes Y to global memory and then reads it back. The fused expression is

$$Z_i = g(f(X_i)).$$

Fusion is legal when there are no intervening dependencies that require materializing Y . The memory traffic reduction is approximately one write and one read of Y :

$$\Delta\text{Bytes} \approx 2Nb,$$

where N is the number of elements and b is bytes per element.

Common Misconceptions

Misconception: Correct Indexing Is Enough

Correct indexing ensures the right elements are accessed, but performance also depends on memory coalescing, reuse, occupancy, and instruction selection. A formally correct scalar kernel can be orders of magnitude slower than a tiled kernel.

Misconception: More Threads Always Means Faster

More threads help hide latency, but after a point, performance is limited by memory bandwidth, compute throughput, register pressure, or shared memory usage. A kernel with too many registers per thread may reduce occupancy and become slower.

Misconception: Tiling Is Only an Optimization Trick

Tiling is not merely a trick. It is a systematic transformation of the iteration space:

$$i = i_o T_i + i_i, \quad j = j_o T_j + j_i, \quad k = k_o T_k + k_i.$$

This transformation exposes locality and parallel structure. It is one of the core bridges between mathematical tensor programs and GPU hardware.

Misconception: Floating-Point Reductions Are Exactly Associative

Floating-point addition is not exactly associative. A parallel reduction may produce a slightly different result than a serial reduction. This is not necessarily a bug, but it must be understood.

Misconception: Shared Memory Automatically Improves Performance

Shared memory helps only when it increases reuse or improves access patterns. If data is loaded into shared memory and used once, the kernel may become slower due to extra instructions and synchronization overhead.

Practical Takeaways

- Start from a precise tensor equation before writing a CUDA kernel.
- Make the iteration domain explicit.
- Derive index mappings from logical indices to blocks, warps, lanes, and registers.
- Treat boundary predicates as part of the kernel specification, not as an afterthought.
- Separate algorithm from schedule.
- Use tiling to expose data reuse.
- Use shared memory only when it reduces global memory traffic or improves access structure.
- Analyze adjacent lane addresses to reason about coalescing.
- Estimate arithmetic intensity before optimizing.
- Track resource usage: registers, shared memory, active warps, and occupancy.
- Remember that reductions require special care because parallel accumulation changes operation order.
- Verify correctness against a simple reference implementation before profiling.

Project or Experiment Ideas

Experiment 1: Derive and Benchmark Vector Addition

Implement vector addition in three forms:

1. PyTorch reference.
2. Naive CUDA kernel.
3. Vectorized CUDA kernel using `float4`.

Measure achieved bandwidth:

$$\text{Bandwidth} = \frac{\text{Bytes moved}}{\text{Time}}.$$

For FP32 vector addition,

$$\text{Bytes moved} \approx 3N \cdot 4.$$

Experiment 2: Naive Versus Tiled Matrix Multiplication

Implement naive and tiled matrix multiplication. Compare:

- runtime;
- achieved FLOPs;
- global memory load efficiency;
- shared memory usage;
- occupancy;
- numerical error relative to PyTorch.

Use

$$\text{FLOPs} = 2MNK$$

and

$$\text{Throughput} = \frac{2MNK}{\text{Time}}.$$

Experiment 3: Boundary Predicate Stress Test

Test matrix sizes that are not multiples of tile sizes, such as

$$M = 1003, \quad N = 997, \quad K = 509.$$

Verify that the kernel does not read or write out of bounds and matches a reference result within tolerance.

Experiment 4: Coalescing Study

Write two kernels for copying a matrix:

- one where adjacent lanes read adjacent columns;
- one where adjacent lanes read adjacent rows.

Measure bandwidth and inspect memory transactions with a profiler.

Experiment 5: Formal Schedule Documentation

For a selected kernel, write down:

- the mathematical specification;
- the iteration domain;
- the tiling transformation;
- the CUDA block mapping;
- the thread mapping;
- the boundary predicates;
- the memory hierarchy plan;
- the expected arithmetic intensity.

Then implement the kernel and compare the measured behavior against the derived expectation.

Final Mental Model

Formalized kernel derivation is the discipline of treating GPU kernel writing as a sequence of semantics-preserving transformations. The starting point is a tensor equation. The next step is an explicit iteration space. Then we choose a schedule: tiling, reordering, parallel mapping, caching, vectorization, and accumulation strategy. Each transformation must preserve correctness while improving the match between the program and the GPU memory hierarchy.

The core mental model is:

A high-performance kernel is not just code. It is a proof-shaped mapping from mathematical computation to hardware execution. The proof gives correctness; the mapping gives performance.

For any kernel, ask five questions:

1. What exact tensor equation is being computed?
2. What is the iteration and reduction domain?
3. How are logical indices mapped to blocks, warps, lanes, and registers?

4. What data is reused, and at which memory level is it stored?
5. What are the limiting resources: bandwidth, compute, registers, shared memory, synchronization, or occupancy?

A kernel derived this way is easier to verify, easier to optimize, and easier to explain to another engineer.